pgvector: Iterative Index Scans

Spiegazioni semplici, esempi SQL e limiti pratici

Il problema che risolvono

Una ricerca con indice approssimato può trovare candidati che vengono poi scartati da WHERE. Le scansioni iterative, disponibili da pgvector 0.8.0, consentono di proseguire l'esplorazione quando i risultati non bastano, entro limiti configurabili.

Il flusso

1. L'indice trova candidati. 2. PostgreSQL applica il filtro. 3. Se servono altre righe, la scansione esplora altro indice. 4. Si ferma quando ha abbastanza risultati oppure raggiunge i propri limiti. Non equivale a ricominciare da zero per ogni tentativo.

```
BEGIN;
SET LOCAL hnsw.iterative_scan = strict_order;

SELECT id, titolo
FROM documenti
WHERE lingua = 'it'
ORDER BY embedding <=> $1::vector
LIMIT 10;
COMMIT;
```

Strict order

Con HNSW, strict_order mantiene l'ordine per distanza dei risultati restituiti. Non garantisce però di aver recuperato tutti i veri migliori vicini: ordinare correttamente i candidati trovati è diverso dal trovarli tutti.

Relaxed order

relaxed_order può fornire un recall migliore, ma ammette risultati leggermente fuori ordine. È disponibile per HNSW e IVFFlat. Per IVFFlat si usa SET ivfflat.iterative_scan = relaxed_order.

Riordinare i risultati relaxed

Puoi materializzare i candidati e riordinarli nella query esterna. Questo corregge l'ordine dei candidati recuperati, non recupera quelli mancanti. L'esempio usa distanza + 0, come indicato dalla documentazione per PostgreSQL 17 e successivi.

```
BEGIN;
SET LOCAL hnsw.iterative_scan = relaxed_order;

WITH candidati AS MATERIALIZED (
  SELECT id, embedding <=> $1::vector AS distanza
  FROM documenti
  WHERE lingua = 'it'
  ORDER BY embedding <=> $1::vector
  LIMIT 20
)
SELECT * FROM candidati
ORDER BY distanza + 0
LIMIT 10;
COMMIT;
```

Limiti e risorse

hnsw.max_scan_tuples limita approssimativamente le tuple visitate nella scansione iterativa; non modifica la scansione iniziale. hnsw.scan_mem_multiplier regola il budget di memoria della scansione rispetto a work_mem. ivfflat.max_probes limita l’espansione delle liste esplorate. Aumentare i limiti può costare latenza e risorse.

Cosa verificare

Le scansioni iterative non garantiscono dieci risultati né recall perfetto. Se il filtro ammette poche righe, la scansione può diventare costosa. Verifica versione installata, piano di esecuzione, recall e latenza con filtri rappresentativi.

Fonte e ambito

Sintesi didattica degli argomenti discussi, basata sulla documentazione ufficiale di pgvector. I parametri mostrati non sostituiscono benchmark sui propri dati. Le funzioni disponibili dipendono dalla versione installata.

https://github.com/pgvector/pgvector/blob/master/README.md